

实 验 报 告

实验名称: 实验八 BP 算法分析与实现

课程名称: 认知科学与类脑计算

实验地点: K2 227

实验日期: 2025/9/12

班 级: 23 级智能班

姓 名: 宋浩宇

学 号: 202300130183

一 实验目的：

加深对 BP 算法的理解，能够使用 BP 算法解决简单问题

二 实验环境：

主系统：Windows 11 家庭中文版 23H2 ; ArchLinux

编辑器：Visual Studio Code ; PyCharm 2025.1.2 ; neovim 0.11

Python 解释器：Python 3.9

Pytorch 版本：2.8.0

CUDA 版本：12.8.97

硬件环境：

CPU：13th Gen Intel(R) Core(TM) i9-13980HX 2.20 GHz

内存：32.0 GB (31.6 GB 可用)

磁盘驱动器：NVMe WD_BLACKSN850X2000GB

显示适配器：NVIDIA GeForce RTX 4080 Laptop GPU

CUDA 核心数：7424

三 实验内容：

根据 BP 算法的相关知识，使用 Python 语言实现一个简单 BP 算法模型。

四 实验步骤：

1. 实验程序：

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))
def sigmoid_derivative(y):
    return y * (1 - y)
x = np.array([[0.35], [0.9]])
target = np.array([[0.5]])
input_size = 2
hidden_size = 2
output_size = 1
learning_rate = 0.5
max_epochs = 100000
np.random.seed(42)
W1 = np.random.randn(hidden_size, input_size) * 0.5
b1 = np.zeros((hidden_size, 1))
W2 = np.random.randn(output_size, hidden_size) * 0.5
b2 = np.zeros((output_size, 1))
for epoch in range(max_epochs):
```

```

z1 = W1 @ x + b1
a1 = sigmoid(z1)
z2 = W2 @ a1 + b2
y = sigmoid(z2)
loss = 0.5 * np.sum((target - y) ** 2)
if loss < 0.01:
    print(f"Converged at epoch {epoch}, loss={loss:.6f},
output={y.ravel()[0]:.6f}")
    break
delta_output = (y - target) * sigmoid_derivative(y)
delta_hidden = (W2.T @ delta_output) * sigmoid_derivative(a1)
dW2 = delta_output @ a1.T
db2 = delta_output
dW1 = delta_hidden @ x.T
db1 = delta_hidden
W2 -= learning_rate * dW2
b2 -= learning_rate * db2
W1 -= learning_rate * dW1
b1 -= learning_rate * db1
print("Final Output:", y.ravel()[0])
print("Final Loss:", loss)

```

2. 程序实验结果:

```

F:\Homework\认知科学与类脑计算作业\.venv\Scripts\python.exe F:\Homework\认知科学与类脑计算作业\实验8\BP.py
Converged at epoch 0, loss=0.000611, output=0.465056
Final Output: 0.46505621995469804
Final Loss: 0.0006105338819272219

进程已结束，退出代码为 0

```

3. 分析和改进:

从结果上来看，BP 算法成功工作，并且效果还不错，如果把训练的 loss 阈值再调小一点可能效果会更好，因为实际上也只跑了一轮就有这个结果了。

五 实验小结:

这次实现的 BP 算法就是最简单的形式，没有计算图，没有自动求导，纯就是手动求了个导然后把这个导函数拿来用了。不过效果还挺好，一轮就能非常近似了。